



Better shifting

Document number:

[P3793R0](#)

Date:

2025-07-15

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Brian Bi <bbi5291@gmail.com>

Co-authors:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3793/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/better-shifting.cow



ABSTRACT: We propose the addition of functions to the `<bit>` header to perform bit shifts on integer operands. The `std::shl` and `std::shr` functions provide the following advantages over the built-in shift operators:

1. They always produce a mathematically correct result when possible.
2. They never have undefined behavior.
3. They avoid the confusing precedence of the built-in shift operators.

§ Contents

- 1 **Introduction**
- 2 **Motivation — Why are overlong shifts needed?**
- 3 **Design considerations**
 - 3.1 **Proposed behavior for overlong shifts**
 - 3.1.1 Wrapping behavior is not useful
 - 3.1.2 Shift amounts which are overlong by more than one should be safe
 - 3.2 **Logical versus arithmetic shifts**



3.3	Negative shift amounts
3.3.1	Why no shift in the other direction?
3.3.2	Why no unspecified result?
3.3.3	Why no undefined behavior?
3.3.4	Conclusion
3.4	Naming
3.5	Signatures
3.6	SIMD support
4	Wording
4.1	[version.syn]
4.2	[bit]
4.3	[simd]
5	References

§ 1. Introduction

C++ has built-in shift operators, `<<` and `>>`, inherited from C with semantics essentially unchanged, including the following two inconvenient properties:

1. The precedence of these operators is less than the precedence of the additive operators, which is counterintuitive because shift operations behave as multiplication and division by a power of 2.
2. If the shift amount is greater than or equal to the width of the (promoted) left operand or is negative, the behavior is undefined. In the remainder of this document, we will refer to shifts by a positive amount that is greater than or equal to the bit width of the operand as *overlong* shifts.

The first property certainly cannot be changed at this point in time. Reflector discussion revealed that GCC relies on the second property when vectorizing multiple shift operations on adjacent memory locations, and changing it might therefore not be "free". Besides that, changing the second property is also likely to be contentious because some committee members prefer to make overlong shifts erroneous behavior instead of defining them to produce the mathematically correct result.

For these reasons, we are instead proposing to solve the problems with shifting in C++ by introducing new Standard Library facilities in the `<bit>` header.

§ 2. Motivation — Why are overlong shifts needed?

We consider the undefined behavior of overlong shifts to be gratuitous. Unlike other arithmetic operators, which produce undefined behavior only when the result is "not mathematically defined or is outside the range of representable values" ([\[expr.pre\] paragraph 4](#)), the built-in shift operators unconditionally produce UB for overlong shifts. This behavior is inherited from

C, in which the arithmetic operators were originally designed to do whatever the corresponding hardware instructions would do; processor families differ as to how overlong shifts are handled. However, it is unclear why the behavior of overlong shifts was standardized as being undefined behavior as opposed to producing an unspecified result; perhaps there's some CPU that we (the authors of this paper) don't know about (and that might even be obsolete) on which the behavior could include trapping, halting, or otherwise failing to produce a result.

Besides that, there are practical reasons why the undefined behavior of overlong shifts is inconvenient, particularly when the second operand is equal to the bit width of the first operand.



EXAMPLE: Consider the task of implementing the following function:

```
/// Return the value of x with the least significant num_bits bits masked out.  
/// num_bits shall be nonnegative and ≤ 32.  
unsigned std::uint32_t mask_lsb(std::uint32_t x, int num_bits);
```

When `num_bits` is equal to 32, the behavior of `mask_lsb` is the natural continuation of the behavior for smaller values of `num_bits`. However, to implement this function, we must guard against `num_bits == 32`:

```
unsigned std::uint32_t mask_lsb(std::uint32_t x, int num_bits) {  
    if (num_bits == 32) return 0;  
    return x & ~((static_cast<std::uint32_t>(1) << num_bits) - 1);  
}
```

The first statement in the body of `mask_lsb` would be unnecessary if the shift operation produced the mathematically correct result of 0 when `num_bits` is equal to 32.

Overlong bit-shifts can also be a problem with right-shifting, although these problems are significantly less common.



EXAMPLE: We can use an integer `b` to represent a bitset, where 1 represents an element which is in the set, and 0 represents an integer that does not.

```
std::uint32_t bitset = /* ... */;  
  
// Possibly undefined behavior; would require special cases to guard against overlong shifts.  
bool contains(size_t i) {  
    return (bitset >> i) & 1;  
}  
  
// Never undefined.  
bool contains(size_t i) noexcept {  
    return std::shr(bitset, int(i)) & 1;  
}
```

Reporting an element that bitset has no capacity for as not being in the set may be exactly the behavior we want, rather than UB, and `std::shr` makes that easy.



§ 3. Design considerations

§ 3.1. Proposed behavior for overlong shifts

We propose the addition of Standard Library functions `std::shl` and `std::shr` that produce mathematically correct results for overlong shifts. That is, these functions actually just shift the bits by the number of positions specified: in the case of an overlong shift, that means that all bits that would be shifted are shifted off the end. For a logical shift, the result is `0`; for an arithmetic right shift, the result is `-1` when the first operand is negative, and `0` otherwise.

§ 3.1.1. Wrapping behavior is not useful

We believe that the "wrapping" behavior (most famously exhibited by the x86 family of processors) in which the shift amount is reduced modulo the bit width of the other operand is not useful, other than when implementing bit rotations. Since the functions `std::rotr` and `std::rotr` are already provided by the Standard Library, C++ programmers do not need to implement rotations themselves anymore, and would not benefit from wrapping behavior for shifts.

§ 3.1.2. Shift amounts which are overlong by more than one should be safe

Although shifts by an amount that are strictly greater than the bit width of the first operand are not as useful as when the amount is equal to the bit width, we believe that requiring the implementation to produce the mathematically correct result in those cases does not impose an additional performance burden. For example, if `x` is 32 bits wide, then requiring `std::shift_left(x, 33)` to produce `0` does not impose additional overhead beyond only requiring `std::shift_left(x, 32)` to produce `0`.



EXAMPLE: For example, GCC 15.1 at `-O2` or higher produces very similar x86-64 assembly for the following two functions:

```
unsigned int shl1(unsigned int x, unsigned int amount) {
    if (amount < 32) return x << amount; else return 0;
}
unsigned int shl2(unsigned int x, unsigned int amount) {
    if (amount > 32) std::unreachable();
    if (amount < 32) return x << amount; else return 0;
}
```

```
shl1:
    mov     ecx, esi
    xor     eax, eax
    sal     edi, cl
```



```

        cmp     esi, 32
        cmovb   eax, edi
        ret

shl2:
        mov     ecx, esi
        xor     eax, eax
        sal     edi, cl
        cmp     esi, 32
        cmovne  eax, edi
        ret

```

Using `shl2` instead of `shl1` can improve performance only in the sense that UB enables more optimizations in general (e.g. assuming that the branch leading to the UB is not taken). The generated code in Clang 20 is very similar.

Such similarities in generated assembly are observed across many other architectures because if a single branch or conditional move instruction is required in order to produce the mathematically correct result for amount ≥ 32 , then a single one will also be needed merely to account for the case of amount $= 32$ (when amount > 32 is disallowed).

§ 3.2. Logical versus arithmetic shifts

The built-in `>>` operator performs an arithmetic shift on signed operands: that is, the sign bit is extended. One possible design is to provide both arithmetic and logical right shifts, either as separate functions or as one function with an additional parameter indicating the choice of arithmetic or logical shift.

However, we believe that this is unnecessary because in cases where the programmer wishes to always perform a logical shift, it is customary to employ unsigned types, possibly by inserting a cast that will be optimized out. Conversely, the deliberate choice to use a signed type for the left operand of a right shift indicates intent to perform an arithmetic shift. The proposed `std::to_signed` and `std::to_unsigned` functions in [P3643R0] would make such conversions convenient, even in generic code.

A survey of popular programming languages supports this design direction:

- Languages that provide an explicit choice between a logical shift or an arithmetic shift usually lack unsigned integer types (Fortran, Java, JavaScript, OCaml). C# had unsigned integer types from its initial release, but didn't have the logical right shift operator until version 11.
- Languages that do not provide an explicit choice between a logical shift and an arithmetic shift always perform arithmetic shifts for negative operands (Go, Haskell, Objective-C, Python, Ruby, Rust).
- Perl is possibly an exception to the above, but it is hard to track down the behavior of shifts prior to version 5.

§ 3.3. Negative shift amounts

The behavior of negative shift amounts in some popular programming languages is listed below:



- Shift amount reduced modulo bit width of other operand: C#, Java, JavaScript
- Exception or panic: Go, Haskell, Python
- Shift in other direction: Fortran, Objective-C, Perl, Ruby
- Unspecified or implementation-defined result: OCaml, Rust

Unlike overlong shifts, there is no obvious "mathematically correct result" for a shift by a negative number of bits, nor is there an obvious choice based on existing practice in other programming languages. Hardware instructions interpret the shift amount as an unsigned number.

There are several options for how the proposed `std::shl` and `std::shr` functions could handle negative shift widths:

1. Shift in other direction.
2. Implementation-defined result.
3. Erroneous behavior with implementation-defined result (proposed).
4. Unspecified result.
5. Erroneous behavior with unspecified result.
6. Undefined behavior.

§ 3.3.1. Why no shift in the other direction?

The first option imposes performance overhead compared with only handling overlong shifts, which makes the second and third options more attractive. Use cases where we want to shift in the other direction are extremely uncommon, and in those uncommon use cases, these operations can be easily expressed by conditionally shifting to the left or right.

§ 3.3.2. Why no unspecified result?

Hardware architectures don't always guarantee a specific result for bit-shifting instructions. For example, older Intel x86-64 processors do not guarantee that shifting wraps, but yield an unspecified result. Therefore, it might seem at first that requiring a predictable, implementation-defined result imposes additional overhead compared with leaving the result unspecified. However, all hardware that we are aware of treats the shift amount as an unsigned integer: that is, any attempt to shift by a negative amount would be interpreted as an attempt to shift by a large positive amount congruent to the original negative amount. Since we already propose to require shifting by large positive amounts to produce a defined result, an implementation could simply define negative shifts to produce the same result as overlong shifts without incurring an additional performance penalty:

```
unsigned shr(unsigned x, int s) {  
    // Handles negative, not just over-large shifts, at no extra cost.  
    return unsigned(s) >= width-v<T> ? 0u : x >> s;  
}
```

§ 3.3.3. Why no undefined behavior?



Unlike division by zero, there is little motivation to make this operation undefined; the underlying hardware instruction does not raise an exception, but produces a surprising (possibly unspecified) result.

If the programmer wants shifting by a negative amount s to have undefined behavior, they can opt into that using:

```
if (s < 0) std::unreachable();  
// or  
[[assume(s >= 0)]];
```

§ 3.3.4. Conclusion

Attempting to shift by a negative amount is often a bug, with the programmer's intent being unclear. However, there is no necessity for undefined behavior.

Therefore, we propose the third option (erroneous behavior with implementation-defined result).

§ 3.4. Naming

We chose the names `shl` and `shr` because they are as short as possible, while still being familiar to many programmers. For example, the x86 and ARM instruction sets have shift instructions named `SHL` and `SHR`, Pascal has built-in operators with these names, and Rust uses `Shl` and `Shr` as the names of the traits that must be implemented for types to support the shift operators.

We hope the brevity will encourage adoption of `std::shl` and `std::shr` as safe alternatives to the built-in operators. These abbreviations are also consistent with the existing `std::rotl` and `std::rotr` functions, which are conspicuously not named `std::rotate_left` and `std::rotate_right`, respectively.

§ 3.5. Signatures

We propose the following signatures:

```
template<class T>  
constexpr T shl(T x, int s) noexcept;  
  
template<class T>  
constexpr T shr(T x, int s) noexcept;
```

The rationale for these signature is as follows:

- We follow the design of `std::rotl` and `std::rotr` in proposing that the shift functions do not perform integer promotion.

- Like `rotl` and `rotr`, these functions don't participate in overload resolution unless `T` is an integer type. Unlike `rotl` and `rotr`, the proposed `shl` and `shr` accept either signed or unsigned integral types for the first operand: for `shr`, the signedness of the first parameter determines whether an arithmetic or logical shift is performed, and it would be surprising if `shl` did not also accept signed types.
- We follow the design of `std::rotl` and `std::rotr` in specifying a type of `int` for the second parameter. `int` is large enough to hold any realistic shift width, and the choice of a signed type rather than an unsigned type allows the implementation to catch bugs in which the expression for the second argument produced a negative result.
- Although the Lakos rule states that Standard Library functions with narrow contracts should not be declared `noexcept`, we don't have precedent for functions that never produce undefined behavior but produce erroneous behavior for some subset of arguments. However, we think that we can sidestep that debate because of the precedent that functions that are closely analogous to built-in operators that are not potentially-throwing should be declared `noexcept` even if they have narrow contracts. If that is the case, it should be even more the case for functions that sometimes exhibit erroneous behavior.

§ 3.6. SIMD support

Following [P2933R4], almost all `<bit>` functions have a corresponding overload in `<simd>`, including `std::rotl` and `std::rotr`.

The proposed functions should also have `<simd>` overloads, in the style of `simd::rotl` and `simd::rotr`. That is, overloads that can either shift a `simd::vec` by a `simd::vec` of shift amounts, or by a scalar shift amount which applies to all elements in the `simd::vec`.

§ 4. Wording

The following changes are relative to [N5008].



NOTE: The `shl` and `shr` functions should immediately precede the `rotl` and `rotr` functions in the wording. Therefore, if this paper and [P3764R0] are adopted in the same meeting, the `shl` and `shr` functions should be inserted between the `msb_to_mask` and `rotl` functions.

§ 4.1. [version.syn]

Bump feature-test macros in [version.syn] as follows:



```
#define __cpp_lib_bitops 201907L 20XXXXL // freestanding, also in <bit>
#define __cpp_lib_simd 202502L 20XXXXL // also in <simd>
```

§ 4.2. [bit]



In [bit.syn], change the synopsis as follows:



```
namespace std {  
    [...]  
  
    // [bit.shift], shifting  
    template<class T>  
        constexpr T shl(T x, int s) noexcept;  
    template<class T>  
        constexpr T shr(T x, int s) noexcept;  
  
    // [bit.rotate], rotating  
    template<class T>  
        constexpr T rotl(T x, int s) noexcept;  
    template<class T>  
        constexpr T rotr(T x, int s) noexcept;  
  
    [...]  
}
```

In [bit], add a new subclause immediately preceding [bit.rotate]:



Shifting

[bit.shift]

```
template<class T>  
    constexpr T shl(T x, int s) noexcept;
```

Constraints: T is a signed or unsigned integer type ([basic.fundamental]).

Effects: If s is negative, the behavior is erroneous and an implementation-defined value is returned. Otherwise, returns the value of x multiplied by 2^s .

[Note: If s is greater than or equal to the bit width of T, the result is zero. — end note]

```
template<class T>  
    constexpr T shr(T x, int s) noexcept;
```

Constraints: T is a signed or unsigned integer type ([basic.fundamental]).

Effects: If s is negative, the behavior is erroneous and an implementation-defined value is returned. Otherwise, returns the value of x divided by 2^s , rounded toward negative infinity.

[Note: If s is greater than or equal to the bit width of T, the result is -1 if x is negative and 0 otherwise; the computation of 2^s does not overflow. — end note]

§ 4.3. [simd]

In [\[simd.syn\]](#), change the synopsis as follows:



```
[...]

// [simd.bit], Bit manipulation

[...]

template<simd-type V0, simd-type V1>
    constexpr V0 shl(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V  shl(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 shr(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V  shr(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotl(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V  rotl(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotr(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V  rotr(const V& v, int s) noexcept;

[...]

// See [simd.bit], Bit manipulation
using simd::byteswap;
using simd::bit_ceil;
using simd::bit_floor;
using simd::has_single_bit;
using simd::shl;
using simd::shr;
using simd::rotl;
using simd::rotr;
using simd::bit_width;
using simd::countl_zero;
using simd::countl_one;
using simd::countr_zero;
using simd::countr_one;
using simd::popcount;

[...]
```

In [\[simd.bit\]](#), immediately preceding the first declaration of `rotl`, insert the following:



```
template<simd-type V0, simd-type V1>
    constexpr V0 shl(const V0& v0, const V1& v1) noexcept;
```



```
template<simd-type V0, simd-type V1>
constexpr V0 shr(const V0& v0, const V1& v1) noexcept;
```

Constraints:

- The type `V0::value_type` is a signed or unsigned integer type ([basic.fundamental]),
- the type `V1::value_type` models integral,
- `V0::size() == V1.size()` is true, and
- `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is true.

Returns: A `basic_simd` object where the i^{th} element is initialized to the result of `bit-func(v0[i], static_cast<int>(v1[i]))` for all i in the range `[0, V::size())`, where `bit-func` has the same behavior as the corresponding scalar function from `<bit>` except that the type of its second parameter is considered to be `V1::value_type`.

```
template<simd-type V>
constexpr V shl(const V& v, int s) noexcept;
template<simd-type V>
constexpr V shl(const V& v, int s) noexcept;
```

Constraints: The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Returns: A `basic_simd` object where the i^{th} element is initialized to the result of `bit-func(v[i], s)` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

§ 5. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P2933R4] Daniel Towner, Ruslan Arutyunyan. Extend `<bit>` header function with overloads for `std::simd` (2025-02-13)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2933r4.html>

[P3643R0] Jan Schultke. `std::to_signed` and `std::to_unsigned` (2025-03-13)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3643r0.html>

[P3764R0] Jan Schultke. A utility function for propagating the most significant bit (2025-07-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3764r0.html>